#1 Method `insert` takes a treap instance, the key and the priority to insert; the method doesn't return anything but it has side effects. Duplicates are allowed (added to a node's left subtree).
#2 Initialize two temporary variables, for current node (initially the treap's root) and its parent.
#3 Create a new node for the key and priority passed (just out of convenience, we create it in a single place).
#4 Traverse the tree until you get to a **null** node (when this happens, **parent** will point to a leaf).
#5 Update **parent**, since current node is not **null**.
#6 Check how the new key compares to current node's key: if it's not larger, we take the left branch.
#7 Otherwise, we go right.
#8 Now we are outside the `while` loop, so `node==null`, but we also need to check that parent is not `null`: it will be null only if the root of the tree is itself `null`, i.e. if the tree is empty.
#9 If the treap was empty, we never entered the while loop, and we just need to create a new root; once that's assigned to the treap's internal field, we are finished.
#10 We need to check if we should add the new key as the left or right child of `parent`.
#11 Either way, we need to set the right parent link for the newly added node.
#12 Now we need to check heap's invariants: until they are violated, or we get to the root, we need to bubble up current node.
#13 If this node is a left child, then we need to use `rightRotate`.
#14 Otherwise, we will rotate `newNode` left.
#15 In case at the end of the cycle `newNode` bubbled up to the root, we need to update the `root` property of the treap.